

Dokumentacja inżynierii wymagań

Raport zawiera przykładowe elementy wykonanej dokumentacji inżynierii wymagań.

Projekt: "SafeGate" – System Kontroli Dostępu 2FA (QR + Biometria)

1. Zespół projektowy i kompetencje

Zdefiniowano zespół niezbędny do realizacji zadania obejmującego wizję komputerową oraz aplikację webową

Kompetencje Techniczne	Michał Szczepański	Filip Kucharski	Mateusz Bem
Frontend (React, TypeScript, Tailwind CSS)	Podstawy	Posiada	Nie posiada
Backend (Python, Flask, FastAPI)	Posiada	Podstawy	Posiada
Bazy Danych (sqlite, postgresql)	Posiada	Podstawy	Podstawy
Znajomość UML	Posiada	Posiada	Posiada
Machine Learning	Podstawy	Podstawy	Posiada
Testowanie oprogramowania	Posiada	Posiada	Podstawy
Wizja komputerowa	Nie posiada	Podstawy	Posiada
Konteneryzacja (Docker, Docker Compose)	Podstawy	Podstawy	Posiada
Kontrola Wersji (Git)	Posiada	Posiada	Posiada

2. Uszczegółowienie wymagań (Faza analityczna)

Poniższa tabela prezentuje wyniki analizy problemu biznesowego i przekłada je na techniczne aspekty rozwiązania.

Pytanie analityczne	Odpowiedź / Decyzja projektowa	Uwagi techniczne i ryzyka
Jaki jest model weryfikacji?	Weryfikacja 1:1 (One-to-One). Kod QR wskazuje tożsamość, system sprawdza tylko, czy twarz pasuje do tego konkretnego ID.	Znacznie szybsze i mniej zasobożerne niż przeszukiwanie całej bazy (1:N). Wymaga < 5 sek. na proces.
Jakie są parametry wejściowe obrazu?	Obraz z kamery USB/IP/wbudowanej urządzenia. Konwersja klatki wideo do formatu Base64 w momencie wykrycia QR.	Wymagana odpowiednia rozdzielczość dla zapewnienia skuteczności >90%.
Co robimy w przypadku braku twarzy w kadrze?	System odrzuca wejście. Raportuje błąd: "Brak twarzy".	Należy zaimplementować detekcję twarzy (face detection) przed etapem rozpoznawania (recognition).
Jak traktujemy fałszywy QR?	Jeśli QR nie istnieje w bazie lub wygasł -> Natychmiastowa odmowa bez uruchamiania kamery.	Oszczędność zasobów obliczeniowych (CPU/GPU). Raportujemy tylko "Zły kod QR".
Jaka jest polityka retencji danych?	Logi i raporty przechowywane przez 6 miesięcy. Po tym czasie automatyczne usuwanie (Auto-purge).	Zgodność z RODO. Zdjęcia z prób wejścia (opcjonalne) powinny być szyfrowane.
Skalowalność systemu?	Startowo 20 osób.	Baza danych SQL (PostgreSQL).

Czy wszystkie wymagania klienta są możliwe do spełnienia? TAK

3. Format danych wejściowych i model danych

3.1 Struktura Bazy Danych (Uproszczona)

Tabela: employees (Pracownicy)	Typ danych	Opis
id	INT (PK, Auto)	Unikalny identyfikator systemowy
full_name	VARCHAR(100)	Imię i nazwisko pracownika
face_encoding	VECTOR(128)	Wektor cech biometrycznych (nie samo zdjęcie JPG)
qr_token	UUID (String)	Unikalny ciąg zaszyty w kodzie QR
qr_valid_until	DATETIME	Data ważności przepustki
reference_photo	TEXT (Base64)	Zdjęcie poglądowe do panelu admina
reference_photo	TEXT	Zdjęcie referencyjne do wyświetlenia w panelu admina
created_at	TIMESTAMPTZ	Data utworzenia konta pracownika

Tabela: access_logs(Raport)	Typ danych	Opis
log_id	INT (PK)	ID zdarzenia
timestamp	TIMESTAMP	Data i godzina próby wejścia
employee_id	INT (FK)	Powiązanie z pracownikiem (NULL jeśli QR nieznany)
status	ENUM	SUCCESS, FACE_MISMATCH, INVALID_QR, NO_FACE
confidence	FLOAT	Poziom pewności dopasowania (np. 0.94)
captured_image	TEXT	Zakodowane zdjęcie z próby wejścia
qr_content	TEXT	Zeskanowany kod QR

Tabela: admins	Typ danych	Opis
id	INT	ID administratora
username	TEXT	Nazwa użytkownika administratora
hashed_password	TEXT	Hash hasła
role	TEXT	Rola (default admin)
last_login	TIMESTAMPTZ	Data ostatniego logowania
created_at	TIMESTAMPTZ	Data utworzenia konta

3.2 Strumień Danych Wejściowych (Real-time)

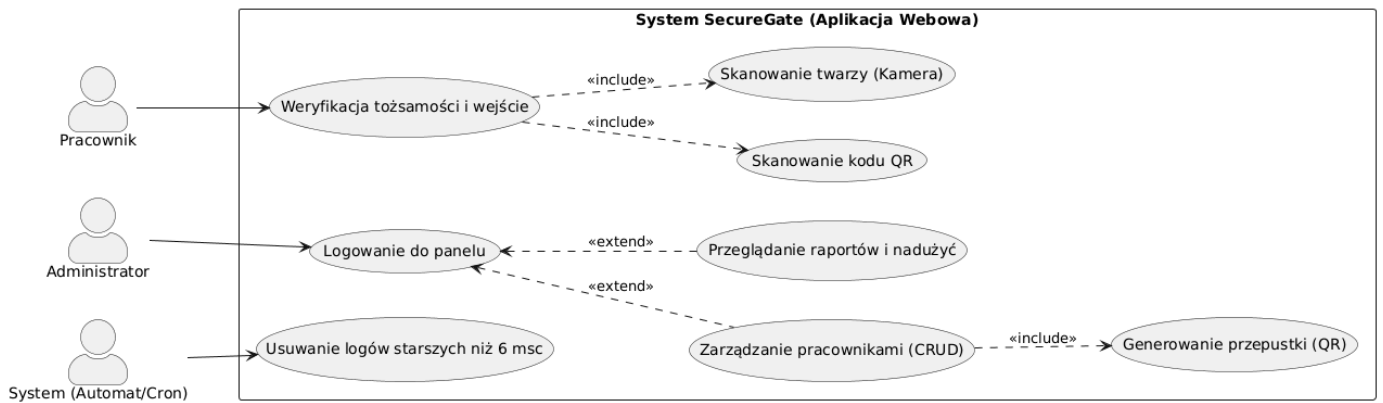
1. Input 1 (QR Scanner): Ciąg znaków (String).
2. Input 2 (Camera): Klatka wideo (Frame) -> przekształcona do cv2.image -> enkodowana do Base64.

4. Modelowanie systemu

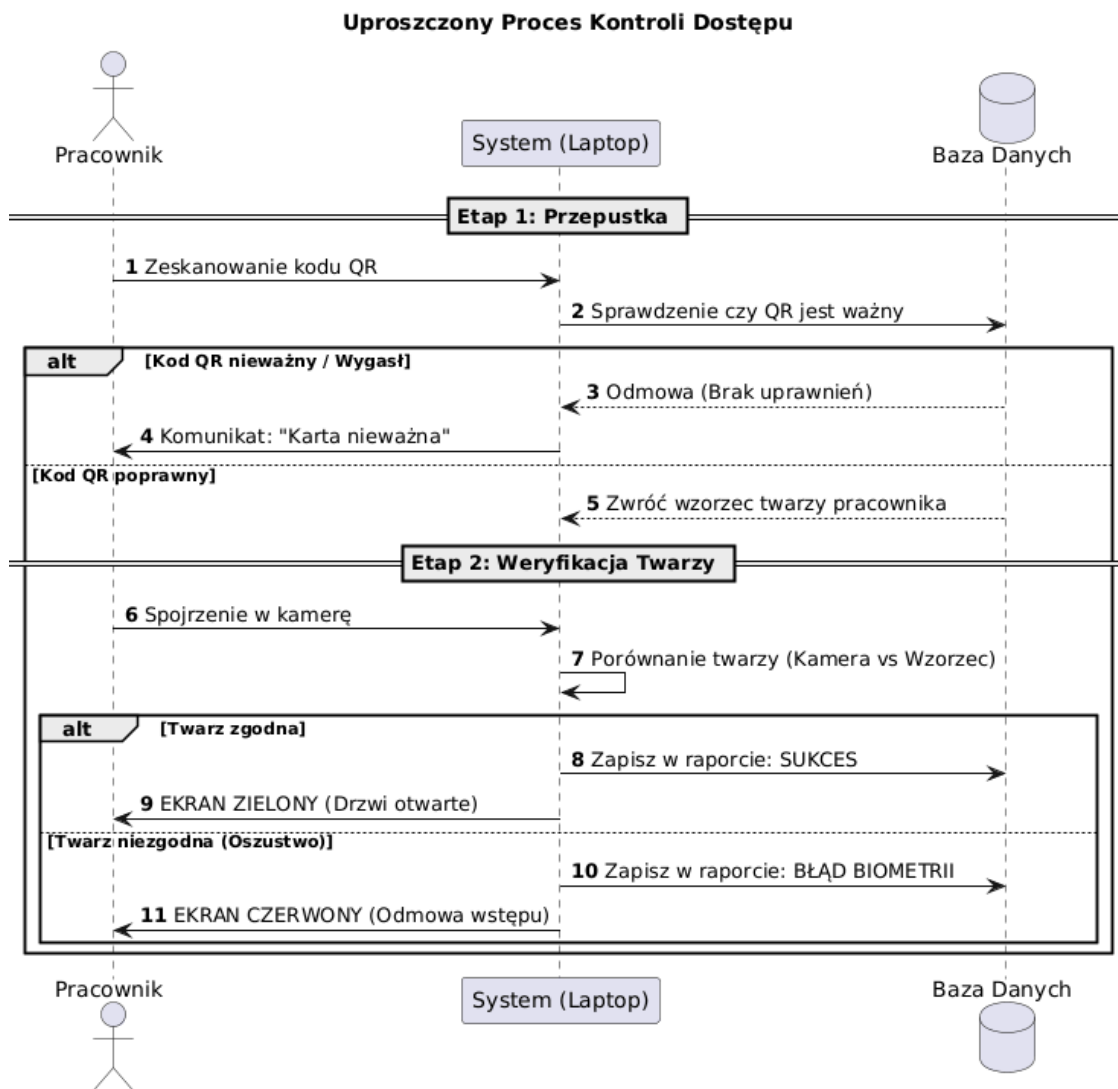
4.1 Karta Systemu

Element	Opis
Aktorzy	Pracownik (User), Administrator (Security/HR)
Cel	Automatyczna weryfikacja tożsamości w celu eliminacji przekazywania kart dostępowych.
Wyzwalacz	Zeskanowanie kodu QR przez czytnik/kamerę kiosku wejściowego.
Dane wejściowe	Token z kodu QR + Obraz twarzy "na żywo".
Warunki brzegowe	Czas przetwarzania < 5s. Skuteczność > 90%.
Odpowiedź	Komunikat wizualny (Zielony/Czerwony ekran) + Rekord w bazie raportów.

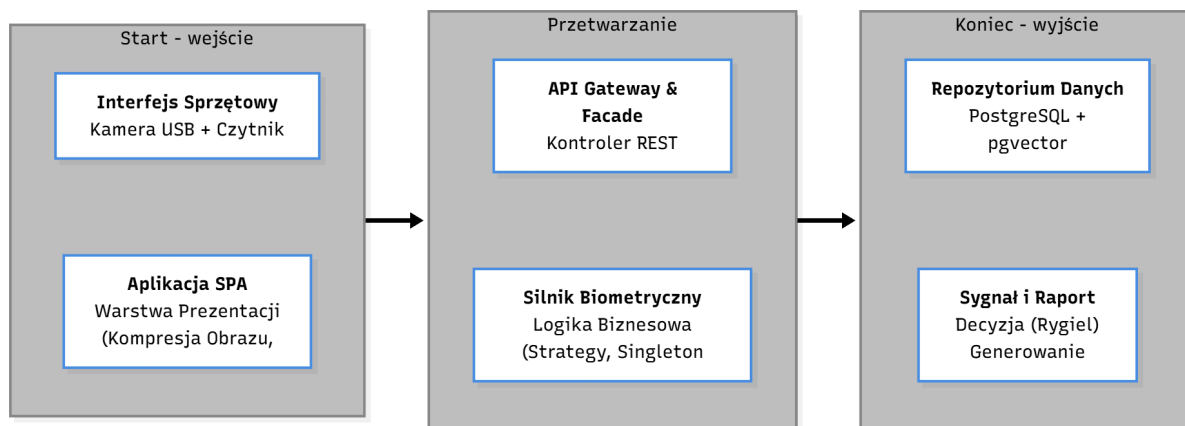
4.2 Przedstawienie modelowanego systemu za pomocą diagramów UML.



5. Diagram sekwencyjny UML



6. Schemat projektowanej architektury



7. Architektura Systemu i Decyzje Projektowe

Niniejszy rozdział definiuje wysoko poziomą strukturę systemu **SecureGate** oraz kluczowe wzorce projektowe, które zostaną zastosowane w procesie implementacji w celu zapewnienia jakości, skalowalności i łatwości utrzymania oprogramowania.

7.1. Styl Architektoniczny

System zostanie zrealizowany w oparciu o **Architekturę Warstwową (Layered Architecture)**. Decyzja ta podyktowana jest koniecznością fizycznej separacji logiki interfejsu użytkownika, logiki biznesowej weryfikacji oraz warstwa trwałego przechowywania danych. Podejście to zapewnia modularność systemu oraz umożliwia niezależne testowanie poszczególnych komponentów.

Struktura systemu składa się z trzech głównych warstw:

1. **Warstwa Prezentacji (Presentation Layer)**: Aplikacja typu Single Page Application (SPA), odpowiedzialna za interakcję z użytkownikiem, podgląd wideo z kamery oraz wyświetlanie komunikatów dostępu.
2. **Warstwa Logiki Biznesowej (Business Logic Layer)**: Realizowana jako API REST (backend), zawierająca kontrolery sterujące przepływem informacji oraz silnik biometryczny przetwarzający obraz.
3. **Warstwa Danych (Data Access Layer)**: Odpowiada za komunikację z bazą danych PostgreSQL, abstrakcję zapytań SQL oraz obsługę operacji wektorowych.

7.2. Zastosowane Wzorce Projektowe

W celu rozwiązania typowych problemów inżynierskich związanych z przetwarzaniem obrazu i dostępem do sprzętu, a także aby zapewnić wysoką jakość kodu (*Maintainability*) i jego elastyczność (*Extensibility*), wdrożone zostaną następujące wzorce projektowe:

Wzorzec (Pattern)	Komponent docelowy	Uzasadnienie zastosowania
Repository Pattern	Warstwa Danych (DAL)	Służy do abstrakcji warstwy dostępu do danych. Jego kluczowym zadaniem jest izolacja logiki aplikacji od specyficznych zapytań wektorowych bazy PostgreSQL (rozszerzenie <code>pgvector</code>), co umożliwia łatwą wymianę silnika bazy danych w przyszłości bez ingerencji w logikę biznesową.
Facade Pattern	Moduł Przetwarzania Obrazu	Hermetyzuje złożoność bibliotek niskopoziomowych (OpenCV, dlib, NumPy). Wystawia uproszczony, spójny interfejs dla warstwy API, ukrywając szczegóły techniczne takie jak konwersja przestrzeni barw (BGR do RGB) czy normalizacja histogramu.
Singleton Pattern	Połączenie do bazy danych	Gwarantuje istnienie wyłącznie jednej instancji obiektu zarządzającego połączeniem do bazy danych.

8. Zakres MVP

Priorytet	Funkcjonalność / Wymaganie	Uzasadnienie
MUST HAVE	Weryfikacja tożsamości i wejście (QR + Twarz)	Główny cel systemu. Musi działać automatycznie zgodnie z diagramem sekwencji.
MUST HAVE	Logowanie sukcesów i porażek do bazy	Niezbędne do audytu bezpieczeństwa i debugowania błędów biometrii.
MUST HAVE	Sygnal sterujący rygłem	System musi wystawić sygnał otwarcia drzwi (czyli poprawnego uwierzytelnienia użytkownika i przepuszczenia go dalej)
SHOULD HAVE	Panel Administratora (GUI)	Przeglądanie logów jest ważne, ale w pierwszej fazie admin może sprawdzać wyniki bezpośrednio w bazie SQL.
SHOULD HAVE	Usuwanie logów (Automat/Cron)	Wg diagramu to zadanie automatyczne, ale w MVP logi z kilku dni nie zapchają dysku. Można to robić ręcznie.
COULD HAVE	Generowanie przepustki (PDF/Druk)	Przepustki QR można wygenerować zewnętrznym narzędziem w MVP. Funkcja wbudowana w system to luksus.
WON'T HAVE	Rejestracja zdjęć "na żywo"	Funkcja robienia zdjęć pracownikom przez kamerkę w panelu admina. W MVP wgrywamy gotowe zdjęcia (Base64).

9. Specyfikacja API

Endpoint: Weryfikacja wejścia

POST /api/v1/access/verify

- **Opis:** Główny punkt wejścia. Odbiera token QR i klatkę wideo, zwraca decyzję o otwarciu drzwi.
- **Request Body (JSON):**

```
{
  "qr_token": "550e8400-e29b-41d4-a716-446655440000",
  "image_base64": "data:image/jpeg;base64,/9j/4AAQSkZJRgABAQ...",
  "timestamp": "2024-11-27T10:00:00Z"
}
```
- **Response (200 OK):**

```
{
  "access_granted": true,
  "message": "Witaj, Michał Bober",
  "confidence_score": 0.94,
  "door_unlock_duration_ms": 5000
}
```
- **Response (403 Forbidden):**

```
{
  "access_granted": false,
  "error_code": "FACE_MISMATCH",
  "message": "Weryfikacja biometryczna nie powiodła się."
}
```

Endpoint: Pobranie logów

GET /api/v1/admin/logs?timestamp=XX

- **Opis:** Umożliwia administratorowi pobranie określonej liczby logów bazując od podanego timestampu w formacie 8601 (RRRR-MM-DD GG:MM:SS)

Endpoint: Dodanie użytkownika (użytkowników) do systemu

POST /api/v1/admin/add_user

- **Opis:** Dodaje nowych użytkowników do systemu. Backend automatycznie przetwarza przesłane zdjęcie (reference_photo) na wektor biometryczny (face_encoding) oraz generuje id.

- **Obsługa plików:** Frontend parsuje załączony przez administratora plik JSON i wysyła jego zawartość w users_list.
- **Request Body (JSON):**

```
{
  "users_list": [ {
    "full_name": "Jan Kowalski",
    "qr_token": "550e8400-e29b-41d4-a716-446655440000", // Opcjonalne: jeśli brak,
    backend generuje UUID
    "qr_valid_until": "2025-12-31T23:59:59Z",
    "reference_photo_base64": "data:image/jpeg;base64,/9j/4AAQSkZJRg..." //
    Wymagane do wygenerowania wektora twarzy
  },
  {
    "full_name": "Anna Nowak",
    "qr_token": "auto-generate",
    "qr_valid_until": "2025-06-30T12:00:00Z",
    "reference_photo_base64": "data:image/png;base64,iVBORw0KGgo..."
  }
]
```
- **Response (201 Created):**

```
{
  "success": true,
  "added_count": 2,
  "errors": [],
  "message": "Pomyślnie dodano użytkowników." }
```
- **Response (400 Bad Request):**

```
{
  "success": false,
  "message": "Błąd walidacji danych.",
  "errors": [ "Użytkownik 'Jan Kowalski': Nieprawidłowy format zdjęcia (brak twarzy)." ]
}
```

Endpoint: Modyfikacja danych użytkowników

PUT /api/v1/admin/users

- **Opis:** Aktualizuje istniejące rekordy. Nadpisuje tylko przesłane pola. Pozwala na masową aktualizację (np. "przedłuż wszystkim ważność o miesiąc" poprzez plik JSON).
- **Request Body (JSON):**

```
{
```

```

"users_list": [ { "id": 105, // Identyfikator w bazie
"qr_valid_until": "2026-01-01T00:00:00Z" // Tylko zmiana daty
},
{ "qr_token": "550e8400-e29b-41d4-a716-446655440000", // Alternatywna
identyfikacja po tokenie
"full_name": "Janina Kowalska", // Zmiana nazwiska "reference_photo_base64":
"data:image/jpeg;base64,..." // Aktualizacja wzorca twarzy (wymusi przeliczenie
face_encoding)
}
]
}

```

- **Response (200 OK)**

```

{
  "success": true,
  "modified_count": 2,
  "message": "Zaktualizowano dane użytkowników."
}

```

Endpoint: Usuwanie użytkowników

DELETE /api/v1/admin/users

- **Opis:** Usuwa użytkowników z bazy danych (lub oznacza jako nieaktywnych, zależnie od polityki RODO – tutaj zakładamy usunięcie rekordu).

- **Request Body (JSON)**

```

{
  "ids_to_delete": [105, 106, 120], // Lista ID do usunięcia
  "tokens_to_delete": [] // Opcjonalnie lista tokenów QR
}

```

- **Response (200 OK)**

```

{
  "success": true,
  "deleted_count": 3,
  "message": "Użytkownicy zostali trwale usunięci z systemu."
}

```

Endpoint: Czyszczenie archiwum logów

DELETE /api/v1/admin/logs/prune

- **Opis:** Usuwa trwale z bazy danych logi wejść, które są starsze niż podana data graniczna (cutoff_date).

- **Request Body (JSON)**

```
{
  "cutoff_date": "2024-05-27T00:00:00Z", // Data graniczna (np. dzisiaj minus 6
  miesięcy)
  "confirm": true // Flaga bezpieczeństwa, zapobiega przypadkowemu wywołaniu
}
```
- **Response (200 OK)**

```
{
  "success": true,
  "deleted_count": 850, // Liczba usuniętych rekordów
  "message": "Pomyślnie usunięto logi starsze niż 2024-05-27.",
  "disk_space_freed_mb": 12.5 // Opcjonalne: informacja o zwolnionym miejscu
}
```
- **Response (400 Bad Request)**

```
{
  "success": false,
  "message": "Brak parametru 'confirm' lub nieprawidłowy format daty."
}
```
- **Response (500 Internal Server Error)**

```
{
  "success": false,
  "message": "Błąd bazy danych podczas usuwania rekordów.",
  "details": "Database locked / Foreign key constraint violation" // Przykładowe
  szczegóły
}
```

Endpoint: Logowanie Administratora

POST /api/v1/auth/login

- **Opis:** Weryfikuje poświadczenia administratora (login i hasło). W przypadku sukcesu zwraca token dostępowy (JWT), który musi być dołączany do nagłówka każdego kolejnego zapytania administracyjnego (np. pobierania logów czy dodawania użytkowników).
- **Request Body (JSON)**

```
{
  "username": "admin",
  "password": "super_secret_password123"
}
```
- **Response (200 OK):**

```
{
  "success": true,
```

```
"access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", // Token JWT
"token_type": "Bearer",
"expires_in": 3600, // Czas ważności w sekundach (np. 1 godzina)
"user": {
  "id": 1,
  "role": "admin",
  "last_login": "2024-11-27T09:15:00Z"
}
```

- **Response (401 Unauthorized)**

```
{
  "success": false,
  "message": "Nieprawidłowy login lub hasło."
}
```

- **Response (429 Too Many Requests):**

```
{
  "success": false,
  "message": "Zbyt wiele nieudanych prób logowania. Spróbuj ponownie za 5 minut."
}
```

10. Projekt UI/UX

Wytyczne dla Frontendu. Aplikacja działa w trybie pełnoekranowym i posiada 4 główne stany:

1. **Stan Spoczynku (Idle):**

- Duży, czytelny napis: "Zeskanuj kod QR, aby wejść".
- Animacja/Ikona sugerująca przyłożenie przepustki.
- Podgląd z kamery jest ukryty lub rozmyty (prywatność), aktywuje się dopiero po wykryciu QR.

2. **Stan Przetwarzania (Processing):**

- Wyzwalacz: Wykryto poprawny QR.
- Widok: Podgląd z kamery "na żywo" z ramką szukającą twarzy.
- Komunikat: "Proszę spojrzeć w kamerę..."
- Spinner ładowania weryfikacji.

3. **Stan Sukcesu (Access Granted):**

- Tło: **Intensywna zieleń (#22c55e)**.
- Ikona: Duży "Checkmark".
- Tekst: "Witaj, [Imię Nazwisko]".
- Widoczny przez 3 sekundy, potem powrót do Idle.

4. **Stan Odmowy (Access Denied):**

- Tło: **Intensywna czerwień (#ef4444)**.
- Tekst błędu: "Odmowa dostępu" (bez szczegółów technicznych dla użytkownika, szczegóły tylko w logach).

11. Strategia Obsługi Błędów (Fail-Secure)

Ze względu na charakter systemu (Kontrola Dostępu), przyjmujemy strategię **Fail-Secure** (bezpieczeństwo zamknięcia):

- **Awaria Bazy Danych:** Jeśli API nie może połączyć się z PostgreSQL – drzwi pozostają **ZAMKNIĘTE**. System nie może wpuścić nikogo na podstawie samego QR (cache) w wersji MVP.
- **Błąd Kamery:** Jeśli *cv2.VideoCapture* zwróci pustą klatkę – system wyświetla komunikat "Awaria kamery" i blokuje wejście.
- **Timeout:** Jeśli weryfikacja trwa dłużej niż 5 sekund, proces jest ubijany, a dostęp odmawiany (timeout exception).